



Security Assessment

Final Report



Polymarket DepositWallet BeaconProxy

May 2026

Prepared for Polymarket

Table of Contents

Project Summary	3
Project Scope.....	3
Project Overview.....	3
Protocol Overview.....	3
Assessment Methodology	5
Findings Summary.....	6
Severity Matrix.....	6
Detailed Findings	7
Low Severity Issues.....	8
L-01: User exit-hatch invariant <code>factory.timelockDelay() <= timelock.minDelay()</code> is unenforced on-chain..	8
L-02: <code>optOut()</code> is not idempotent across beacon upgrades.....	11
Informational Severity Issues.....	13
I-01: DepositWallet interface conformance is not enforced at compile time.....	13
I-02: <code>_readImmutableArgs</code> uses magic number 64 instead of <code>_IMMUTABLE_ARGS_LENGTH</code>	14
I-03: Deprecated factory functions retained without consumers.....	15
I-04: <code>_setImplementation()</code> runs the wallet-interface check after the SSTORE and event, wasting gas on rejected upgrades.....	16
Disclaimer	17
About Certora	17

Project Summary

Project Scope

Project Name	Repository Link	Commit Hash	Platform
Polymarket DepositWallet BeaconProxy	https://github.com/Polymarket/deposit-wallet/commits/release/beacon-proxy/	Initial: 28643fd Fix I-01: 92fd2c4 Fix I-02: 6934f29	Solidity

Project Overview

This document describes the security review of **Polymarket DepositWallet BeaconProxy**. The work was undertaken from **May 4th, 2026** to **May 5th, 2026**.

The following contract list is included in our scope:

`src/*`

The team performed a manual audit of all the files in scope. During the manual audit, the Certora team discovered bugs in the code, as listed on the following page.

Protocol Overview

The Deposit Wallet protocol provides deterministic smart-contract wallets used to execute deposit-related actions on behalf of wallet owners. Wallets are deployed and coordinated through **DepositWalletFactory**, which keeps the operational role model: the factory owner controls factory upgrades and admin assignment, admins manage operators and the wallet pause timelock, and operators deploy wallets and relay signed execution batches.

Each **DepositWallet** is an account-like contract with an owner, replay-protected nonce, optional session signers, ERC-1271 signature validation, and a pause-based recovery path. Normal activity flows through **factory.proxy(...)**: an operator submits one or more EIP-712 signed **Batch** objects, and each wallet verifies the batch wallet address, nonce, deadline, and signature before

executing the ordered calls. Signatures may come from the wallet owner or an authorized session signer, with session signers restricted from calling the wallet itself or the beacon.

The audited PR changes the wallet upgrade model. New wallets are no longer individual UUPS proxies controlled by their own owners. Instead, the factory deploys deterministic ERC-1967 beacon proxies bound to a `DepositWalletBeacon` address baked into the factory implementation. The beacon owner, expected to be a timelock, controls the default wallet implementation for all participating wallets through `upgradeTo(...)`. Candidate implementations must expose the expected `walletInterfaceId()` value, giving the beacon an implementation-identity check before accepting upgrades.

Existing UUPS wallets are migrated through an opt-in adapter flow. A wallet owner can sign a batch that uses the legacy upgrade path to install `BeaconForwarder` as that wallet's implementation, provided the adapter has been authorized by the factory. After installation, the original wallet address and storage remain intact, but calls are forwarded to the implementation returned by the beacon. Because `BeaconForwarder` exposes no wallet-owner upgrade function, migrated wallets lose their former self-upgrade authority and follow the beacon-controlled implementation path.

The system preserves the wallet owner's emergency controls. A wallet owner can pause the wallet, wait the factory-configured `timelockDelay`, and then withdraw assets, revoke approvals, revoke session signers, unpause, or manage beacon participation. The current code also supports per-wallet beacon opt-out and opt-in: after the pause timelock, a wallet can pin itself to the current beacon implementation, or later clear that pin and rejoin future beacon upgrades. Thus, the main trust boundary introduced by the migration is the beacon owner/timelock, while the user-facing escape hatch remains implemented at the wallet level.

Assessment Methodology

Our assessment approach combines design level analysis with a deep review of the implementation to ensure that a protocol is secure, economically sound, and behaves as intended under realistic conditions.

At the design level, we evaluate the architecture, the economic assumptions behind the protocol, and the safety properties that should hold independently of a specific chain or environment. This process includes reviewing internal and cross protocol interactions, state transition flows, trust boundaries, and any mechanism that could be exploited to extract value, deny service, or alter core system behavior. At this stage, a focused threat modelling exercise helps identify key attack surfaces and adversarial capabilities relevant to the system. Design level issues often relate to incentive structures, governance implications, or systemic behavior that emerges under adversarial conditions.

Implementation analysis focuses on the concrete behavior of the code within the execution model of the target chain. This involves reviewing the correctness of logic, access control, state handling, arithmetic behavior, and the nuanced behaviors of the chain environment. Familiar classes of vulnerabilities such as reentrancy conditions, faulty permission checks, precision issues, or unsafe assumptions often surface at this layer. These findings require context aware reasoning that takes into account both the code and the architectural intent.

To support this analysis, the codebase is examined through repeated manual passes and supplemented by automated tools when appropriate. High-risk logic areas receive deeper scrutiny, invariants are validated against both design intent and actual implementation, and potential vulnerability leads are thoroughly investigated. Automated techniques such as static analysis, fuzzing, or symbolic execution may be used to complement manual review and provide additional insight.

Collaboration with the development team plays an important role throughout the audit. This helps confirm expected behaviors, clarify design assumptions, and ensure an accurate understanding of the protocol's intended operation. All findings are documented with clear reasoning, reproducible examples, and actionable recommendations. A follow up review is conducted to validate the applied fixes and verify that no regressions or secondary issues have been introduced.

Findings Summary

The table below summarizes the findings of the review, including type and severity details.

Severity	Discovered	Confirmed	Fixed
Critical	-	-	-
High	-	-	-
Medium	-	-	-
Low	2	2	0
Informational	4	4	2
Total	6	6	2

Severity Matrix

Impact	High	Medium	High	Critical
	Medium	Low	Medium	High
	Low	Low	Low	Medium
		Low	Medium	High
Likelihood				

Detailed Findings

ID	Title	Severity	Status
L-01	User exit-hatch invariant <code>factory.timelockDelay() <= timelock.minDelay()</code> is unenforced on-chain	Low	Acknowledged
L-02	<code>optOut()</code> is not idempotent across beacon upgrades	Low	Acknowledged
I-01	DepositWallet interface conformance is not enforced at compile time	Informational	Fixed
I-02	<code>_readImmutableArgs</code> uses magic number 64 instead of <code>_IMMUTABLE_ARGS_LENGTH</code>	Informational	Fixed
I-03	Deprecated factory functions retained without consumers	Informational	Acknowledged
I-04	<code>_setImplementation()</code> runs the wallet-interface check after the SSTORE and event, wasting gas on rejected upgrades	Informational	Acknowledged

Low Severity Issues

L-01: User exit-hatch invariant `factory.timelockDelay() <= timelock.minDelay()` is unenforced on-chain

Severity: Low	Impact: Medium	Likelihood: Low
Files: src/DepositWallet.sol src/DepositWalletFactory.sol	Status: Acknowledged	

Description:

The migration runbook names a single load-bearing invariant for the user emergency-exit hatch:

Confirm the planned beacon owner is a timelock contract (not an EOA) and that `beaconTimelock.delay() >= factory.timelockDelay()` so users have at least the wallet's pause window to react to a malicious upgrade after observing it queued.

The runbook's "Risks and Mitigations" chapter lists this same invariant as the only mitigation for both "beacon owner compromised / acts maliciously" and "user exit hatch removed by a malicious upgrade":

Beacon owner must be a timelock with `delay ≥ factory.timelockDelay`. Users observing a malicious queued upgrade have `≥ timelockDelay` window to `pause()` their wallet, wait their own timelock, and drain via `withdrawERC20/withdrawERC1155`.

If the invariant breaks, those critical-impact risks lose their only mitigation. The wallet's exit gate reads `paused() + factory.timelockDelay()` and never consults the beacon owner's `minDelay`. The factory's `_setTimelockDelay` validates only against `MAX_TIMELOCK_DELAY`. No on-chain check ties the two values together.

Three independent state transitions can break the invariant without reverting:

1. **Deployment** – the owner `Timelock` contract (`0x47EbFAC3353314C788B96CDcbf41daadfE03629C`) was deployed before this rollout with a `minDelay` chosen independently of the factory's existing `timelockDelay`. The Phase-2 factory

upgrade does not re-validate the relationship, so the new implementation can take effect on a factory whose stored `timelockDelay` already exceeds `minDelay`.

2. **Admin path** - `setTimelockDelay()` is `onlyAdmin`. An admin can raise `timelockDelay` past `minDelay` at any time, and nothing prevents it.

3. **Timelock path** - the beacon owner can lower `minDelay` to any value (down to zero) through the timelock's queue.

In any of these states, a paused user waits longer than the queue-and-execute window for a malicious upgrade.

Severity is rated as Low because exploitation requires operator misconfiguration or a compromised beacon owner, and the trust model already places the beacon owner on a timelock. But the impact is escape-hatch failure in the exact scenario the hatch is meant to cover.

Recommendations:

Enforce the invariant at every state transition that could break it.

1. Add a `migrate()` reinitializer to the new factory implementation, called atomically as part of the Phase-2 upgrade. It asserts the invariant at the moment the new code takes effect; if broken, the upgrade transaction reverts and the proxy stays on the old implementation.

```
function migrate() external reinitializer(2) onlyOwner {
    require(
        timelockDelay <= ITimelock(IOwnable(BEACON).owner()).minDelay(),
        TimelockExceedsBeaconDelay()
    );
}
```

1. Phase 2 becomes `factoryProxy.upgradeToAndCall(newImpl, abi.encodeCall(IFactoryV2.migrate, ()))`.

2. Guard `_setTimelockDelay` against the beacon owner's `minDelay` so the admin path cannot raise `timelockDelay` past it.

```
require(
    _timelockDelay <= ITimelock(IOwnable(BEACON).owner()).minDelay(),
```

```
TimelockExceedsBeaconDelay()  
);
```

3. Have the wallet's pause-exit read `min(factory.timelockDelay(), timelock.minDelay())` instead of `factory.timelockDelay()` alone. The user's exit window then auto-shrinks alongside `minDelay`, so a `setMinDelay` lowering through the queue cannot strand a migrated wallet on a stale (larger) delay.

```
uint256 walletDelay = factory.timelockDelay();  
uint256 ownerDelay  = ITimelock(IOwnable(BEACON).owner()).minDelay();  
uint256 effective   = walletDelay < ownerDelay ? walletDelay : ownerDelay;  
require(block.timestamp > paused() + effective, TimelockInsufficientDelay());
```

Customer Response:

Acknowledged.

L-02: optOut() is not idempotent across beacon upgrades

Severity: Low	Impact: Low	Likelihood: Low
Files: src/DepositWalletBeacon.sol	Status: Acknowledged	

Description:

`optOut()` is the wallet's mechanism to unsubscribe from beacon upgrades. Calling it pins the wallet to the current implementation so future `upgradeTo` calls do not reach it. A second call, however, re-reads the current beacon default and overwrites the pin, effectively upgrading the wallet to any beacon upgrade that has happened in the meantime. The NatSpec calls the function "idempotent" which is misleading.

The function reads `defaultImplementation()` (the global slot) and writes that into `pinnedImplementation[msg.sender]`. It does not check the wallet's existing pin. If the beacon default has changed between calls, the second call replaces the original pin with the new default. The trigger sequence:

1. **T0** – wallet calls `optOut()` and pins to implementation **X**.
2. **T1** – beacon owner upgrades to beacon to implementation **Y**.
3. **T2** – wallet owner calls `optOut()` again. Pin is replaced with **Y**. The wallet adopts **Y** despite having explicitly opted out.

Severity is Low because the trigger requires an owner action. The misleading NatSpec, however, can encourage the re-call pattern and the failure mode contradicts the guarantee opt-out provides.

Recommendations:

Make the function actually idempotent – no-op when a pin already exists:

```
function optOut() external {  
    if (pinnedImplementation[msg.sender] != address(0)) return;  
    address current = defaultImplementation();
```

```
pinnedImplementation[msg.sender] = current;  
emit OptedOut(msg.sender, current);  
}
```

Alternatively, a more defensive approach is to revert when already pinned, forcing the caller to explicitly **optIn** first:

```
function optOut() external {  
    require(pinnedImplementation[msg.sender] == address(0), AlreadyOptedOut());  
    ...  
}
```

Customer Response:

Acknowledged.

Informational Severity Issues

I-01: DepositWallet interface conformance is not enforced at compile time

Files:

src/DepositWallet.sol

Description:

`DepositWallet` implements every function in `IDepositWallet` but does not list the interface in its base contracts. Selector-based dispatch from `DepositWalletFactory` and `DepositWalletBeacon` works at runtime so there is no current bug. But the Solidity compiler does not enforce that the implementation stays in sync with the interface. A future rename, signature change or typo (notably on `walletInterfaceId()`, which `DepositWalletBeacon._setImplementation` staticcalls to validate every upgrade candidate) would compile cleanly and only surface at runtime, blocking beacon upgrades until the implementation is republished.

Recommendations:

Add `IDepositWallet` to the `DepositWallet` base list.

Customer Response:

Fixed in PR #63.

Fix Review:

Fix confirmed.

I-02: `_readImmutableArgs` uses magic number 64 instead of `_IMMUTABLE_ARGS_LENGTH`

Files:

src/DepositWallet.sol

Description:

`_readImmutableArgs` allocates its scratch buffer via `new bytes(_IMMUTABLE_ARGS_LENGTH)` but then hardcodes 64 twice inside the inline assembly – once as the offset (`sub(extcodesize(self), 64)`) and once as the copy length. Solidity `constant` state variables are inlined as literals at compile time and are usable inside `assembly { ... }` blocks so referencing `_IMMUTABLE_ARGS_LENGTH` here would produce identical bytecode.

The current implementation is correct, but if the immutable args layout is ever extended (e.g. a third field appended to the proxy bytecode), the constant and the `abi.decode` schema would be updated naturally while the two literal 64s would silently retain their old value.

Recommendations:

Replace both literal occurrences with the named constant so the assembly stays coupled to `_IMMUTABLE_ARGS_LENGTH`.

Customer Response:

Fixed in PR #64.

Fix Review:

Fix confirmed.

I-03: Deprecated factory functions retained without consumers

Files:

src/DepositWalletFactory.sol

Description:

`setImplementation(address)` writes a slot-0 pointer no longer used by `deploy()`. `predictWalletAddress(address, bytes32)` retains its pre-migration selector but ignores its first argument and returns a beacon-proxy address rather than the legacy UUPS prediction.

The team confirmed there are no on-chain or off-chain consumers and that retention is precautionary, with removal planned in a post-migration upgrade.

Both functions remain a risk for future callers. The residual risk is that an integrator reading the contract's ABI or the legacy deployment's interface can call these selectors and be misled by the result. This is specifically the case for the `predictWalletAddress()` overload, where the function returns a different address than the legacy version did rather than rejecting the call.

Recommendations:

Either remove both functions outright or replace their bodies with an explicit revert so any accidental future call fails loudly:

```
error DeprecatedFunction();

function setImplementation(address) external onlyOwner {
    revert DeprecatedFunction();
}

function predictWalletAddress(address, bytes32) external pure returns (address) {
    revert DeprecatedFunction();
}
```

Customer Response:

Acknowledged.

I-04: `_setImplementation()` runs the wallet-interface check after the SSTORE and event, wasting gas on rejected upgrades

Files:

src/DepositWalletBeacon.sol

Description:

`_setImplementation()` writes the new implementation to storage and emits `Upgraded` inside an assembly block and only then calls `_checkWalletInterfaceId()`. If the probe fails, the revert unwinds the writes but the SSTORE and LOG2 gas (~3.5–5k) has already been paid. The inline comment explains the ordering as preserving Solady's verbatim assembly, but `_checkWalletInterfaceId()` is a Solidity-level call outside the block, so hoisting it does not touch the verbatim code.

Recommendations:

Hoist the check to before the assembly block:

```
function _setImplementation(address newImplementation) internal virtual {
    _checkWalletInterfaceId(newImplementation);
    /// @solidity memory-safe-assembly
    assembly { /* Solady's verbatim block, untouched */ }
}
```

Customer Response:

Acknowledged.

Disclaimer

Even though we hope this information is helpful, we provide no warranty of any kind, explicit or implied. The contents of this report should not be construed as a complete guarantee that the contract is secure in all dimensions. In no event shall Certora or any of its employees be liable for any claim, damages, or other liability, whether in an action of contract, tort, or otherwise, arising from, out of, or in connection with the results reported here.

About Certora

Certora is a Web3 security company that provides industry-leading formal verification tools and smart contract audits. Certora's flagship security product, Certora Prover, is a unique SaaS product that automatically locates even the most rare & hard-to-find bugs on your smart contracts or mathematically proves their absence. The Certora Prover plugs into your standard deployment pipeline. It is helpful for smart contract developers and security researchers during auditing and bug bounties.

Certora also provides services such as auditing, formal verification projects, and incident response.